
PROJECT REPORT

Matrix Template Library in C++

SUBMITTED BY

- 1] PATIL KHUSHBU MAHENDRA- 882697
- 2] KHALID VAFA - 882677
- 3] MUHAMMAD ISRAR- 882801

Table of Contents

<i>Problem Statement</i>	3
<i>Proposed Solution</i>	3
<i>Template Class <T></i>	4
<i>Functionalities of Template Matrix Library:</i>	5
1. Addition of two matrices	5
2. Return submatrix of original matrix with specific dimensions	5
3. Return the transpose of the original matrix	5
4. Column major traversing	5
5. Row major traversing	5
6. Matrix multiplication	5
1. Addition of two matrices:	6
2. Return submatrix of original matrix with specific dimensions	6
3. Return the transpose of the original matrix	6
4. Column major traversing	6
5. Row major traversing	6
6. Matrix multiplication	6

Problem Statement:

Implement a templated library handling matrix, vectors, and covectors; where vectors are $n \times 1$ matrices and covectors are $1 \times n$ matrices.

the library must offer operation like:

submatrix: return a matrix that contains a contiguous subrange of rows and columns of the original matrix

transpose: return a matrix with rows and columns inverted

The matrices returned by such operators must share the data with the original matrix, but direct copy of the matrices and vectors must result in a deep copy (sharing a matrix can be obtained by taking a submatrix extending the whole row and column ranges).

Create iterators for the matrix traversing it either in row-major or column-major order. Any matrix should be traversable in either direction regardless of its natural representation order. Any given iterator will traverse only in a given order.

Provide matrix addition and (row-by-column) multiplication. When dealing with multiple multiplications, the library must perform the sequence of multiplications in the most computationally efficient order, i.e., by multiplying matrices contracting the largest dimension first. For example, let A, B, and C be 2×3 , 3×5 , and 5×2 matrices respectively, then the expression $A * B * C$ should be computed as if it were $A * (B * C)$, performing first the multiplication $B * C$ since it is contracting the dimension 5, and then multiplying A by the result, contracting the smaller dimension 3.

Proposed Solution:

We have tried to propose the solution to the above given problem in the following way;

Template Class <T>

We have defined a class named “Matop” with type <T> (template). The type of the class is not known at the start but will be defined dynamically at the time of creating instances of t class “Matop” from main function.

Variables used in this class are as follows,

- Int Row
- Int column
- T *** myMatrix:

Constructor Matop (int m, int n):

In this constructor we are initializing myMatrix array with 0's.

```
20     MatOp(int m, int n)
21     {
22         mrows = m;
23         ncols = n;
24         ObjMatrix = new T **[mrows];
25         for (int x = 0; x < mrows; ++x) {
26             ObjMatrix[x] = new T*[ncols];
27             for(int y = 0; y < ncols; y++) {
28                 ObjMatrix[x][y] = new T();
29             }
30         }
31     }
```

Constructor Matop (int m, int n, T *** result):

```
37     */
38     MatOp(int m, int n, T ***result)
39     {
40         ObjMatrix = result;
41         mrows = m;
42         ncols = n;
43     }
44
```

We have created this constructor for the initialization of our submatrix.

We have following functions inside Matop class:

1. insertat() Function:

```
45 //Setter function for Matrix Data that requires matrix coordinates for value insertion
46 void insertat(int m, int n, T data)
47 {
48     *ObjMatrix[m][n] = data;
49 }
50
```

2. Matop matrix transpose() function :

```
51 // Function to perform transpose of a given matrix
52 MatOp MatrixTranspose()
53 {
54     /*
55     We need result matrix to return the answer of given function. Thats why we need result variable.
56     Apart from that we are just replcing column with row and row with column in the matrix index.
57     */
58     int m = ncols, n = mrows;
59     MatOp<T> result(m, n);
60
61     for (int x = 0; x < m; x++)
62         for (int y = 0; y < n; y++)
63             result.ObjMatrix[x][y] = ObjMatrix[y][x];
64     return result;
65 }
66
67
68
```

3. Deepcopy () function :

```
69 /* This function basically creates deep copy of the reference object.
70 That means there are more than one matrix objects holding same data.
71 */
72 MatOp DeepCopy()
73 {
74     MatOp<T> result(mrows, ncols);
75     for (int x = 0; x < mrows; x++)
76         for (int y = 0; y < ncols; y++)
77             result.insertat(x, y, *ObjMatrix[x][y]);
78     return result;
79 }
80
```

4. Fetchfrom() of type T;

```

168 //Getter function to get the particular index value from given matrix
169 T fetchfrom(int m, int n)
170 {
171     return *ObjMatrix[m][n];
172 }
173

```

Functionaries of Template Matrix Library:

This library takes matrices as input and do the following operations of matrices.

1. Addition of two matrices.
2. Return submatrix of original matrix with specific dimensions.
3. Return the transpose of the original matrix.
4. Column major traversing.
5. Row major traversing.
6. Matrix multiplication.

1. Addition of two matrices:

In this library we have created a “Static Add Method” with type “Static Matop <T>”, which takes two square matrices as input and iterate through parameters A&B matrices and add their values in new matrix called Result matrix.

2. Return submatrix of original matrix with specific dimensions:

We have created a function named “subMatrix” to which we assign the reference of the original matrix and only limit the column and row values.

3. Return the transpose of the original matrix:

We have created a function named “transposeMatrix” which traverse through the values of the current matrix and swap rows to columns while storing in new matrix.

4. Column major traversing:

This function named “columnMajorTraversing” performs column major traversing and gives a matrix with respect to column major as output.

5. Row major traversing:

This function named “rowMajorTraversing” performs row major traversing and gives a matrix with respect to row major as output.

6. Matrix multiplication:

We have created a method named “static Matop<T> multiply” which takes two matrices as input and check the dimension first that whether these two matrices are multipliable and then performs multiplication. We have also did overloading of “static Matop <T> multiply”, this method takes three matrices as input instead of two matrices and it checks the dimensions whether the multiplication is even possible or not.

In this we have created a temp matrix of type Matop <T>, and another matrix result of type

Matop <T>. This checks that if the columns of A - Matrix is greater than or equal to B - Matrix. If yes, then it performs multiplication of A & B matrices first and store their result in temp matrix and then multiply the temp matrix with C matrix and store it in result matrix i.e., $(R = (A * B) * C)$. And if not, then it performs multiplication of B & C matrices first then store their result in temp matrix and multiply it with A - matrix and store it in result matrix i.e., $(R = (B * C) * A)$.